

COMPTE RENDU D'EXAMEN PRATIQUE

Simulation Arduino sous Proteus ISIS

Détection analogique, feu tricolore, commande
d'éclairage et contrôle angulaire d'un servomoteur.

Filière

Systèmes Automatisés et Intelligence Artificielle pour l'Industrie 4.0

Nom et prénom

Jorf Moaad

Encadrant

M. Aymane BOUMEZZOUGH

Année universitaire

2026–2027

Introduction et méthode

Ce travail étudie quatre fonctions classiques d'un système embarqué : acquérir une grandeur analogique, générer une séquence temporelle, lire une commande tout-ou-rien et positionner un actionneur. L'Arduino Uno assure le traitement, tandis que Proteus ISIS permet de vérifier virtuellement le câblage et le comportement avant un montage réel.

Objectifs du TP

ACQUISITION

Exploiter `analogRead()` et interpréter une mesure codée sur 10 bits, de 0 à 1023.

COMMANDE

Piloter des sorties numériques avec `digitalWrite()` et des temporisations.

INTERFACE

Lire un bouton stabilisé par une résistance pull-down et reproduire son état.

ACTIONNEMENT

Convertir une mesure en angle avec `map()` et commander un servomoteur.

Logiciels et démarche

Outil	Rôle
Arduino IDE	Écriture, compilation et export du programme destiné à l'Arduino Uno.
Proteus ISIS	Saisie du schéma, chargement du fichier compilé et simulation électrique.

1. Câbler le schéma



2. Compiler le code



3. Charger le fichier



4. Simuler et relever

Règles communes de câblage

- Toutes les masses **GND** doivent être communes.
- Chaque LED doit être protégée par une résistance, typiquement **220 Ω**.
- Les broches et polarités indiquées dans l'énoncé sont conservées.
- Le programme compilé doit correspondre exactement au schéma simulé.

Détecteur de seuil analogique

Objectif. Allumer la LED rouge uniquement lorsque la valeur du potentiomètre lue sur A0 est strictement supérieure à 512.

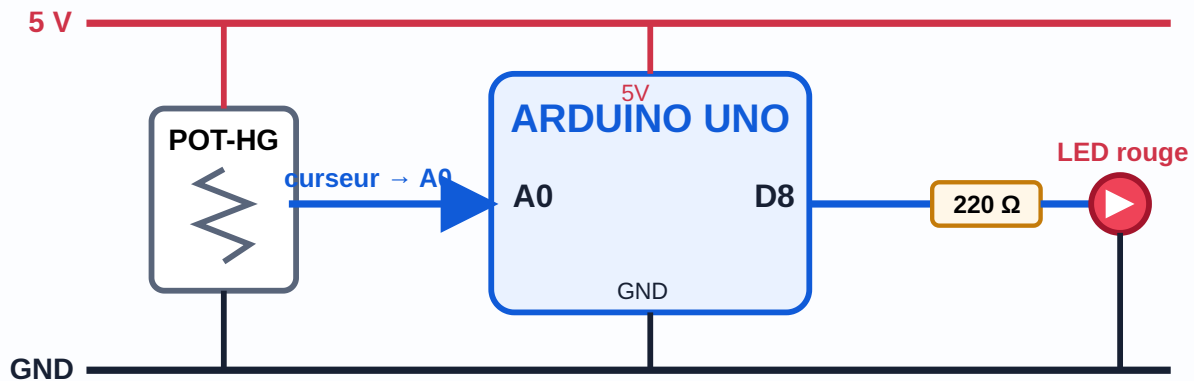
Arduino Uno R3

POT-HG

LED-RED

220 Ω

SCHÉMA DE CÂBLAGE À REPRODUIRE SOUS PROTEUS ISIS



Le curseur du potentiomètre va sur A0. L'anode de la LED est commandée par D8 à travers 220 Ω ; la cathode va à GND.

Élément	Connexion	Rôle
Potentiomètre	Extrémités à 5 V et GND ; curseur à A0	Produit une tension variable de 0 à 5 V.
LED rouge	D8 → 220 Ω → anode ; cathode → GND	Indique le dépassement du seuil.

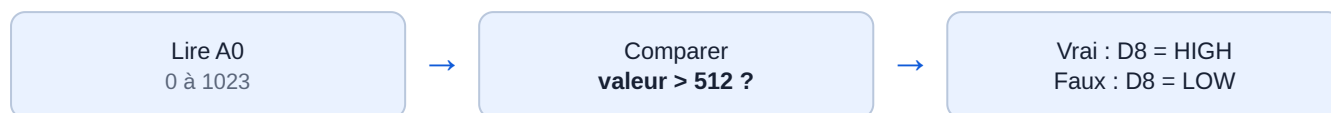
Programme Arduino

```
const byte BROCHE_POTENTIOMETRE = A0;
const byte BROCHE_LED = 8;
const int SEUIL = 512;

void setup() {
  pinMode(BROCHE_LED, OUTPUT);
}

void loop() {
  const int valeur = analogRead(BROCHE_POTENTIOMETRE);
  if (valeur > SEUIL) {
    digitalWrite(BROCHE_LED, HIGH);
  } else {
    digitalWrite(BROCHE_LED, LOW);
  }
}
```

Résultats et analyse



Relevés de validation

Valeur sur A0	Tension approximative	Condition	État attendu de la LED
0	0,00 V	0 > 512 : faux	Éteinte
512	2,50 V	512 > 512 : faux	Éteinte
513	2,51 V	513 > 512 : vrai	Allumée
1023	5,00 V	1023 > 512 : vrai	Allumée

Interprétation. La transition doit se produire entre 512 et 513. À la valeur 512, la LED reste éteinte puisque l'énoncé impose « strictement supérieure ». La moitié théorique de l'alimentation vaut environ 2,5 V.

Conclusion de l'exercice 1

Le montage transforme une tension analogique en une information logique. L'entrée A0 numérise la position du potentiomètre sur 10 bits, puis le programme compare cette mesure au seuil 512. Le résultat attendu est une LED éteinte jusqu'à 512 inclus et allumée à partir de 513. Cet exercice met en évidence l'importance de l'opérateur de comparaison choisi pour définir précisément la frontière de commutation.

Points à vérifier dans Proteus

- ☐ Le potentiomètre est alimenté entre 5 V et GND, sans borne flottante.
- ☐ Le curseur, et non une extrémité, est relié à A0.
- ☐ La LED possède une résistance en série et sa polarité est correcte.
- ☐ La valeur 512 maintient bien la LED éteinte.

Gestion d'un feu tricolore

Objectif. Réaliser automatiquement et indéfiniment la séquence vert 3 s, jaune 1 s, rouge 3 s, avec une seule LED active à la fois.

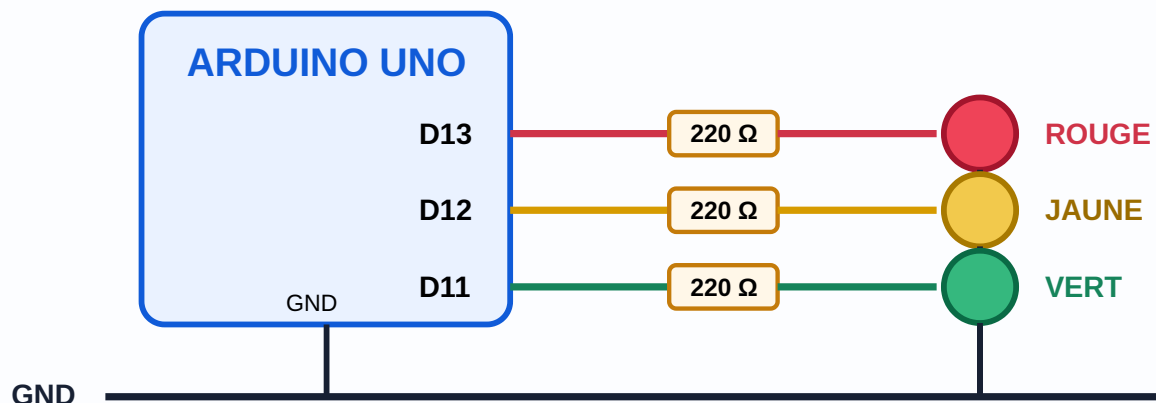
Arduino Uno R3

3 LED

3 × 220 Ω

D11 / D12 / D13

SCHÉMA DE CÂBLAGE À REPRODUIRE SOUS PROTEUS ISIS



Chaque sortie attaque l'anode de sa LED à travers une résistance de 220 Ω. Les trois cathodes sont reliées à la masse.

Couleur	Broche	Connexion
Rouge	D13	D13 → 220 Ω → anode LED ; cathode → GND
Jaune	D12	D12 → 220 Ω → anode LED ; cathode → GND
Verte	D11	D11 → 220 Ω → anode LED ; cathode → GND

Principe temporel



Durée d'un cycle complet : 3 + 1 + 3 = 7 secondes, puis reprise immédiate au vert.

Cycle automatique

Programme Arduino

```
const byte BROCHE_ROUGE = 13;
const byte BROCHE_JAUNE = 12;
const byte BROCHE_VERTE = 11;

void afficherFeu(bool rouge, bool jaune, bool vert) {
    digitalWrite(BROCHE_ROUGE, rouge ? HIGH : LOW);
    digitalWrite(BROCHE_JAUNE, jaune ? HIGH : LOW);
    digitalWrite(BROCHE_VERTE, vert ? HIGH : LOW);
}

void setup() {
    pinMode(BROCHE_ROUGE, OUTPUT);
    pinMode(BROCHE_JAUNE, OUTPUT);
    pinMode(BROCHE_VERTE, OUTPUT);
    afficherFeu(false, false, false);
}

void loop() {
    afficherFeu(false, false, true); delay(3000);
    afficherFeu(false, true, false); delay(1000);
    afficherFeu(true, false, false); delay(3000);
}
```

La fonction `afficherFeu()` écrit explicitement l'état des trois sorties à chaque étape. Cette organisation évite de laisser allumée la couleur précédente et rend le programme plus lisible.

Chronogramme de validation

Intervalle	D13 rouge	D12 jaune	D11 verte	Observation
0 à < 3 s	LOW	LOW	HIGH	Vert seul
3 à < 4 s	LOW	HIGH	LOW	Jaune seul
4 à < 7 s	HIGH	LOW	LOW	Rouge seul
À 7 s	LOW	LOW	HIGH	Nouveau cycle

Conclusion de l'exercice 2

Le programme réalise un cycle routier répétitif de 7 secondes. Les états sont exclusifs : vert pendant 3 s, jaune pendant 1 s, puis rouge pendant 3 s. L'exercice montre comment organiser une séquence déterministe à l'aide de sorties numériques et de `delay()`. Cette méthode convient à ce montage simple, même si elle bloque temporairement le microcontrôleur.

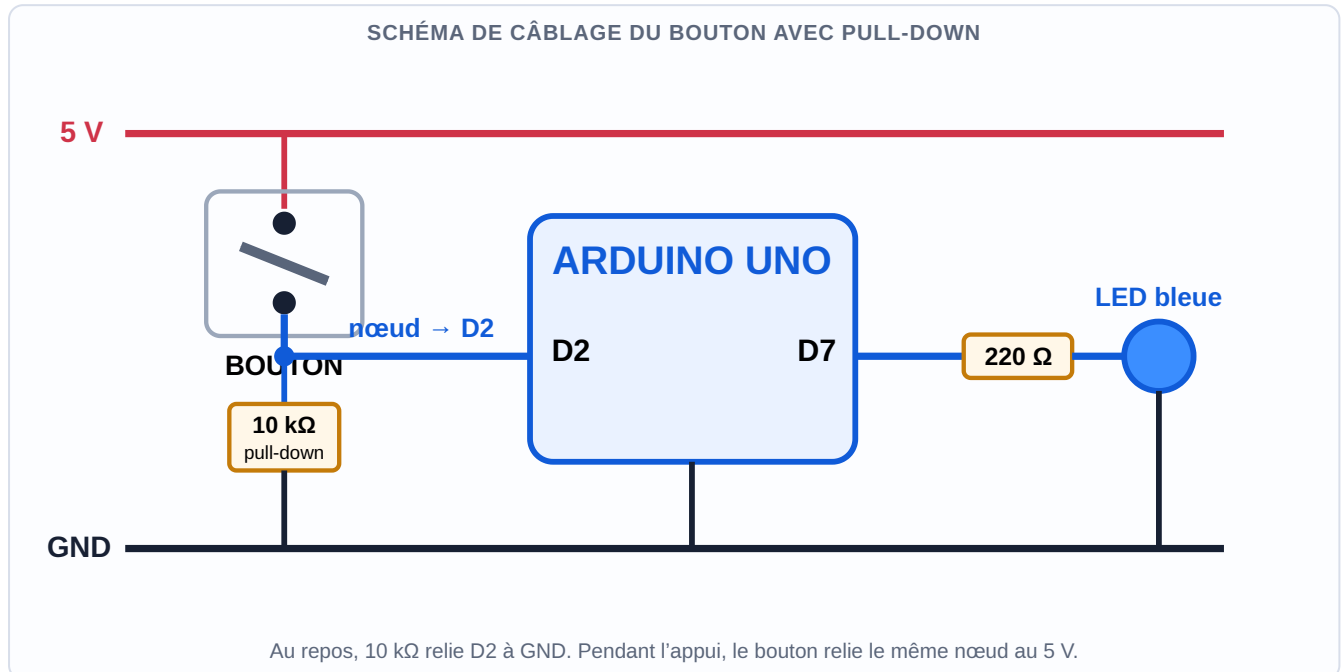
Commande manuelle d'éclairage

Objectif. Maintenir la LED bleue allumée uniquement pendant l'appui sur le bouton. Une résistance de 10 k Ω impose un état bas stable lorsque le bouton est relâché.

Arduino Uno R3

BUTTON

LED-BLUE

Pull-down 10 k Ω 

Pourquoi la résistance pull-down est nécessaire

BOUTON RELÂCHÉ

LOW

La résistance ramène D2 à 0 V. L'entrée ne flotte pas et la LED reste éteinte.

BOUTON APPUYÉ

HIGH

Le bouton impose environ 5 V sur D2. La LED s'allume pendant toute la durée de l'appui.

Élément	Connexion
Bouton	Une borne au 5 V ; l'autre au nœud D2.
Pull-down	10 k Ω entre le nœud D2 et GND.
LED bleue	D7 → 220 Ω → anode ; cathode → GND.

Lecture du bouton

Programme Arduino

```
const byte BROCHE_BOUTON = 2;
const byte BROCHE_LED = 7;

void setup() {
  pinMode(BROCHE_BOUTON, INPUT); // pull-down externe
  pinMode(BROCHE_LED, OUTPUT);
}

void loop() {
  const bool boutonAppuye =
    (digitalRead(BROCHE_BOUTON) == HIGH);
  digitalWrite(BROCHE_LED, boutonAppuye ? HIGH : LOW);
}
```

La broche D2 est déclarée en INPUT parce que la polarisation basse est fournie par une résistance externe de 10 kΩ. Il ne faut pas utiliser INPUT_PULLUP avec ce câblage : cela activerait une polarisation interne opposée et modifierait la logique demandée.

Table de vérité

État du bouton	Tension D2	digitalRead(D2)	Commande D7	LED
Relâché	≈ 0 V	LOW	LOW	Éteinte
Maintenu	≈ 5 V	HIGH	HIGH	Allumée
Relâché de nouveau	≈ 0 V	LOW	LOW	Éteinte

Résultat attendu. Il s'agit d'une commande non maintenue : l'état lumineux suit directement l'état mécanique du bouton. Aucune mémorisation n'est réalisée par le programme.

Conclusion de l'exercice 3

La LED bleue suit l'état du bouton : elle s'allume pendant l'appui et s'éteint dès le relâchement. La résistance de 10 kΩ donne un niveau LOW défini en l'absence d'appui et empêche les commutations aléatoires d'une entrée flottante. L'exercice permet de comprendre la relation entre le câblage de polarisation d'une entrée et la logique du programme.

Contrôle angulaire d'un servomoteur

Objectif. Lire le potentiomètre sur A0, convertir la plage 0–1023 en un angle de 0–180° et transmettre cette consigne au servomoteur connecté à D9.

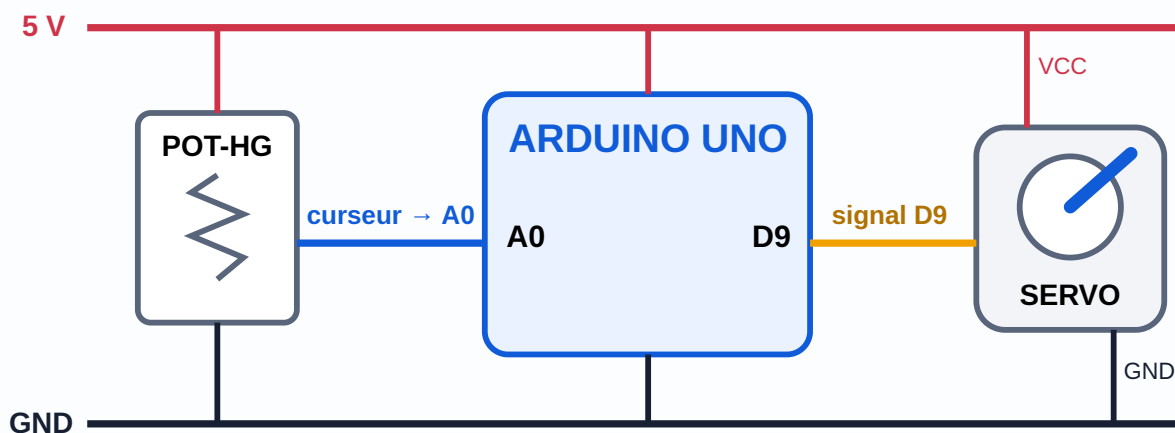
Arduino Uno R3

POT-HG

MOTOR-SERVO

Servo.h

SCHÉMA DE CÂBLAGE DU SERVOMOTEUR



Le potentiomètre et le servo partagent le 5 V et la masse de l'Arduino ; le signal de commande du servo est relié à D9.

Élément	Connexion
Potentiomètre	Extrémités à 5 V et GND ; curseur à A0.
Servomoteur	Signal à D9 ; VCC à 5 V ; GND à la masse commune.

Montage réel. Un servomoteur peut demander plus de courant que la sortie 5 V de la carte ne peut en fournir. En pratique, utiliser si nécessaire une alimentation 5 V régulée externe et relier impérativement sa masse à celle de l'Arduino. Cette précaution n'empêche pas la simulation Proteus demandée.

Conversion mesure–angle

Programme Arduino

```
#include <Servo.h>

const byte BROCHE_POTENTIOMETRE = A0;
const byte BROCHE_SERVO = 9;
Servo servomoteur;

void setup() {
  servomoteur.attach(BROCHE_SERVO);
}

void loop() {
  const int valeur =
    analogRead(BROCHE_POTENTIOMETRE);
  const int angle =
    map(valeur, 0, 1023, 0, 180);
  servomoteur.write(angle);
  delay(15);
}
```



Relation de conversion et points de contrôle

La conversion est linéaire : $\text{angle} \approx \text{valeur} \times 180 / 1023$. La fonction `map()` effectue un calcul entier.

Valeur A0	Position relative	Angle commandé attendu
0	0 %	0°
256	≈ 25 %	≈ 45°
512	≈ 50 %	≈ 90°
768	≈ 75 %	≈ 135°
1023	100 %	180°

Conclusion de l'exercice 4

La position du potentiomètre devient une consigne angulaire continue entre 0° et 180°. La bibliothèque Servo produit le signal de commande nécessaire, tandis que `map()` adapte simplement les deux plages. Le point médian, environ 512, doit positionner l'axe près de 90°. Cet exercice relie ainsi acquisition analogique, mise à l'échelle et commande d'un actionneur.

Conclusion générale

Les quatre montages couvrent une chaîne de traitement complète : acquisition d'une information, décision ou conversion par programme, puis commande d'un indicateur ou d'un actionneur.

EXERCICE 1

Une mesure analogique est comparée à un seuil précis.
La valeur frontière 512 reste dans l'état éteint.

EXERCICE 2

Trois sorties sont coordonnées dans une séquence exclusive et répétitive de 7 secondes.

EXERCICE 3

Une entrée numérique est stabilisée par pull-down et commande directement une LED.

EXERCICE 4

La plage 0–1023 est convertie en 0–180° pour piloter la position d'un servomoteur.

Problèmes possibles et solutions

Symptôme	Cause probable	Correction
Une LED ne s'allume pas	Polarité inversée, résistance mal raccordée ou mauvaise broche.	Vérifier anode/cathode, continuité vers GND et numéro de broche.
La mesure A0 ne varie pas	Curseur du potentiomètre non relié à A0 ou alimentation incomplète.	Relier les extrémités à 5 V/GND et le curseur à A0.
Le bouton commute seul	Entrée D2 flottante ou pull-down mal placé.	Mettre 10 k Ω entre le nœud D2 et GND.
Le servo ne suit pas	Signal, alimentation ou masse commune manquante.	Contrôler D9, VCC et GND ; vérifier Servo . h.
Proteus ne démarre pas le programme	Fichier compilé non chargé ou mauvaise fréquence.	Associer le bon fichier à l'Uno et conserver les paramètres du modèle.

Checklist avant remise

- ☐ Relire l'introduction, les analyses et les difficultés rencontrées.
- ☐ Exporter au format PDF sous le nom Jorf_Moaad_Examen_TP_Arduino.pdf.
- ☐ Utiliser l'objet de mail : Examen TP Arduino – Jorf Moaad.